

Oracle Physical Storage Structures

↳ Physical database Structure:

⇒ The Oracle databases uses a number of physical storage structures on disk to hold and manage the data from transactions.

⇒ Some of these storage structures such as:

Datafiles: Hold actual user data.

Redo log files: Hold user transaction logs

Archived redo: Archived transaction logs

Control files: Maintain the state of the database objects.

Text-based alert trace files: Contains logging information for routine events and errors.

(I) Data files

Definition:

⇒ Every Oracle database must

→ are the OS files that store data within database.
→ Oracle reads and writes these files. They form so that other programs don't read or write it.

⇒ Contains actual data of database
tables, indexes, data dictionary and rollback segments
⇒ datafile composed of → Oracle db blocks
⇒ block size range 4KB to 32KB or blocks ^{small} memory areas ^{area or} block where ^{that things} are stored in db.
Contain at least one datafile.
⇒ Datafiles are like containers that hold data.

⇒ Each datafile corresponds to one physical operating system file on disk.

⇒ Datafiles are members of tablespaces.

⇒ Bigfile tablespace consist of exactly one datafile.

⇒ However, a tablespace consist of many datafiles.

Expansion and Size Management:

⇒ Datafiles may automatically expand if created with the AUTOEXTEND parameter.

⇒ Expansion can be limited using the MAXSIZE.

⇒ Size is ultimately limited by the disk volume.

Functionality / what their job?

⇒ They store all important data of users permanently.

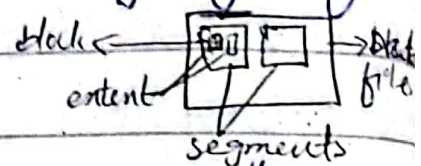
⇒ Frequently accessed blocks are cached in memory.

⇒ When to make change in data files, those changes are first written

Oracle blocks are made up of multiple OS blocks.
 ⇒ Block size → minimum amount of data that can be read or written at one time.
 depends on OS used.
 down in redo log files before they are saved permanently in datafiles.

Datafile structures

- Header: first block of each datafile.
- ^{contains meta data} Contains the critical information that maintains the overall integrity of the database.



Checkpoint structures

- a logical timestamp in the header
 - Indicates the last point at which changes are made to the datafiles
 - Determines which redo logs apply.
- ⇒ DB BLOCK SIZE → initialization parameters → sets the default block size of the db

(II) Control files

Definition

- o Every Oracle database has at least one control file.
- o It maintains metadata about the structure physical of the database.
- o Control file is a small binary file.
- o Control file is the parent file of the database.

⇒ All copies of control files → stored at multiple locations and should be synchronised.

↑ security, privileges, definitions related information.

Contents

o The Control file includes:

⇒ The database name, Creation date and file locations.

⇒ Names and locations of associated datafiles and redo log files.

⇒ Checkpoint information.

⇒ The current log sequence number & Tablespaces information.

⇒ Stores information used by Recovery manager (RMAN) for backups.

o Extension of Control file is .CTL.

o By default Oracle database has 2 Control files.

Multiplexing

o Control files can be multiplexed for redundancy.

o Only one control file is designated as primary for metadata retrieval.

Backup and Recovery

o alter database backup control file to trace command generates a SQL script for recreating the control file.

⇒ Used in case of system failure when all multiplexed binary version of the control file are lost.

Real time updates:

- Changes to the database structure are immediately reflected in control files.

Flexible maintenance:

- Trace file can be used to recreate the control file for renaming databases or changing certain limits without recreating the entire database.

Location:

- Location of control files is defined by CONTROL_FILES initialization parameters.

(3) Redo Log files:

Functionality:

→ Redo log files are filled with redo records.

⇒ A redo record, also called a redo entry

⇒ whenever data is added, removed, or changed in Oracle objects or table

→ Oracle internal activities
→ store in redo log files

, an entry is written to the current redo log file.

minimum requirements

- o At least two redo log files are required per Oracle database.

- o Oracle uses redo log files in a circular fashion.

Management

- o When one redo log file is filled with redo log entries, marked as ACTIVE if it is still needed for instance recovery.

- o INACTIVE if it is not needed for instance recovery.

- o The next file in the sequence becomes CURRENT.

Role in recovery

→ used to play back the changes lost
→ used for "undo" operation

⇒ Ideally the information in redo log file is never used.

⇒ Essential for instance recovery after failures like power outages or server failures.

⇒ Entries in redo log files are applied to database datafiles for roll-forward recovery.

Redundancy:

- multiple copies of redo log files can exist on different physical disks

for resilience and to protect from disk failures and disasters. Oracle writes synchronously to all redo log members.

Multiplexing:

- Redo log files, archived log files, and control files can be multiplexed to ensure database availability and data integrity.

Types of Redo-log files



↓
Online redo logs

- Lives changing are made.

↓
Archived redo logs

- Copies of the filled online redo logs made to avoid losing redo data as the online redo logs are overwritten.

⇒ Each Oracle instance uses a thread of redo to record the changes it makes to the database.

⇒ A thread of redo is composed of redo log groups, which are composed of one or more redo log members.

Q. What is difference between datafiles and tablespaces?

How Oracle uses redo logs?

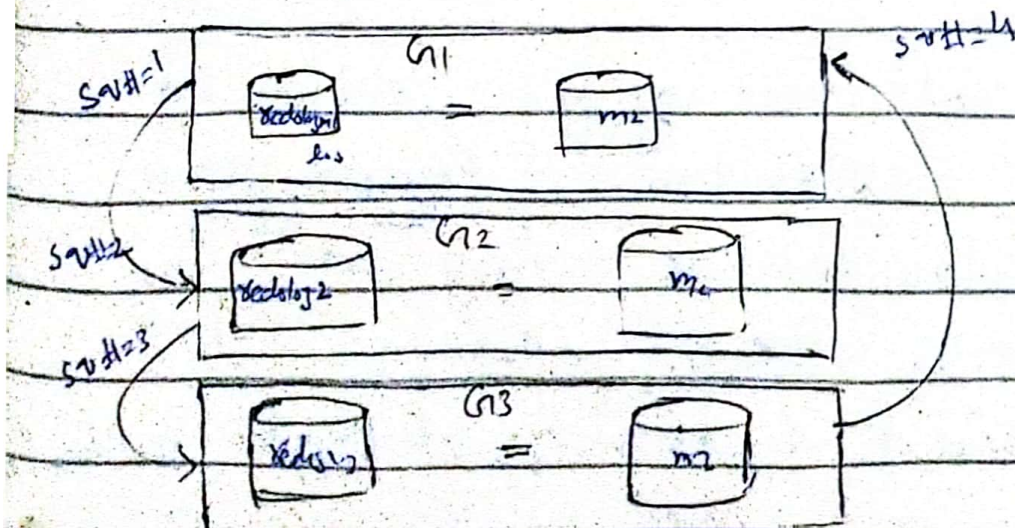
⇒ Once Oracle fills one redo logfile, it automatically begins to use the next logfile.

⇒ Whenever the server cycles through all the available redo log files, it returns to the first one and reuses it.

⇒ Oracle keeps track of the different redo logs by using a redo log sequence number.

⇒ OS uses redo log files to identify the physical file, while Oracle uses the redo logfile sequence numbers to determine the order in which the logs were filled and cycled.

Cycling redo logs:



Archived Log files:-

⇒ An Oracle database can operate in one of two modes:

1. Archive log
2. Non-archivelog

Non-archivelog

⇒ Redo log files are reused circularly.

⇒ Also known as online-redo log files.

⇒ Redo entries (the contents of the previous transactions) are no longer available in case of a failure to a disk drive or another media failure.

⇒ Committed but unwritten transactions available in online redo log files during failures.

Cons: where a failure could lead to unrecoverable data.

Archive log files

⇒ Filled redo log files sent to specified destinations. - Prevents gaps in the history

⇒ Allows database recovery at any point after a media failure.

⇒ Recent backups of datafiles and

redo log files used for recovery.

Oracle data Guard:-

⇒ Also known as Oracle standby database.

⇒ Enhances high availability of facilitating database recovery and continuity.

Naming Conventions for redo logs:

⇒ OS names for the various files that makeup a database are very important for humans to identify these files by their names.

Ex: - redo log m1.log, redologm2.log,

• The prefix and suffix indicate that this is redo log information.

format

⇒ The format for archive redo log names is specified using parameters:

- LOG_ARCHIVE_FORMAT = ORCL %t %s %r

• %t - thread number

• %s - sequence number

• %r - resetlogs ID

Parameters files

Definitions: → are guidelines for manage store and start the db.

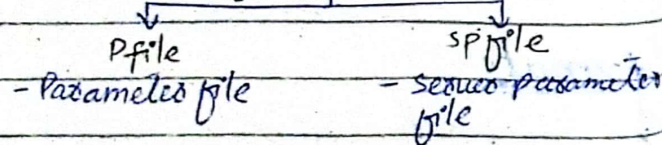
⇒ Parameter file is a text or binary to store the database initialization parameter.

⇒ Tells how database will be start and how database will be shut down.

⇒ These files initialize the the database parameters.

⇒ The oracle instance reads the parameter file during startup which are then used to control the behavior of database instance and many other aspects as well.

Types of parameter file



PFILE:

⇒ Stands for the parameter file.

⇒ It is a text file which can be modified by a text editor.

⇒ It stores data in human readable format.

⇒ With the help of PFILE Oracle server start an Oracle instance.

⇒ Used for manually changing in database

⇒ Older way

⇒ Not recommended by Oracle.

SPFILE:

⇒ Stands for the server parameter file.

⇒ Stores data in binary format.

⇒ Can not be modified by text-editor.

⇒ Created from PFILE.

⇒ Newer way

⇒ Recommended by Oracle.

Startup Sequence

⇒ Instance searches for SPFILE first then PFILE if SPFILE not found.

⇒ Alternatively, startup command can specify PFILE explicitly.

What parameter file can do?

⇒ Initialization parameter files, regardless of the format.

⇒ Can specify file location for:

- trace files
- Control files
- filled redo log files and so forth.

⇒ They also set limits on the sizes of various structures in the SGA as well as how many users can connect to the database simultaneously.

File VS Spfile

File

- stands for parameter file.
- File is a text file.
- Can be edited.

Spfile

- stands for server parameters file.
- spfile is a binary file.
- cannot be edited.

by text editor.

by text editor

- Default name is

- Default name is

init <SID>.ora.

spfile <SID>.ora

- Can be resides

- Can be resides.

on server and client.

on server.

- Read by db server.

- Can be written and read by db server.

Pre-Oracle9i:

⇒ init.ora file was the sole means for specifying initialization parameters.

Drawbacks:

- Manual editing required

Oracle9i onwards:-

⇒ SPFILE introduced for easier parameter management.

⇒ Dynamic parameter changes automatically reflected in SPFILE.

⇒ No need for manual editing:

SPFILE Backup:

⇒ Cannot mirror SPFILE directly, but can back up to init.ora file.

⇒ Both init.ora and SPFILE should be backed up using OS commands or Recovery Manager.

Default SPFILE Creation:

⇒ Database Configuration Assistant (DBCA) creates SPFILE by default during database creation.

Q. How to Create SPFILE in Oracle?

To create an SPFILE in Oracle, the following steps should be followed:

1. Check Current Parameter file type

⇒ Determine whether your database is currently using a PFILE or an SPFILE.

⇒ This can be checked by querying the parameter SPFILE in SQL*PLUS:

```
SELECT value FROM v$parameter  
WHERE name = 'spfile';
```

2. Create Initialization parameter file:

⇒ If it doesn't already have an SPFILE, can be created from existing PFILE or by specifying initialization parameters directly.

⇒ To create an SPFILE from an existing PFILE, use the CREATE SPFILE SQL command:

```
CREATE SPFILE FROM PFILE  
'<path to your pfile>';
```

3. Verify SPFILE Creation:

⇒ After executing the CREATE SPFILE command, check if SPFILE has been successfully created and is available in the default location.

```
SHOW PARAMETER spfile;
```

4. Restart Database Using SPFILE:

⇒ Once the SPFILE is created,

you can use it to start the database.

⇒ If the database is currently running using a PFILE, then need to shut it down first.

⇒ Then start it using the newly created SPFILE.

```
SHUTDOWN IMMEDIATE;
```

```
STARTUP;
```

5. Confining database startup:

⇒ After starting the database using the SPFILE, verify that it has started without errors and that the parameters from the SPFILE are being used:

SELECT name, value FROM V\$parameters.

6. Backup SPFILE:

⇒ Regularly backup SPFILE to ensure copy is available in case of any issues.

⇒ Can be copied manually or automatically.